# Leveraging NAIRR for Research: Infrastructure and Hands-On Workshop

Session 1 of 12 — NAIRR Workshop Series

Oakland University & Worcester Polytechnic Institute

March 21, 2026 • 11:00 AM – 2:00 PM ET

# Today’s Agenda

- Why NAIRR matters for your research (~15 min)
- NAIRR infrastructure deep dive (~15 min)
- Allocation landscape (~10 min)
- Live walkthrough: NAIRR Portal + Jetstream2 (~10 min)

**— Hands-On —**

- Connecting to Jetstream2 (~15 min)
- Jupyter Notebook Crashcourse (~20 min)
- Exercise A: NAIRR Models + HuggingFace (~25 min)
- Exercise B: Bulk Data Processing at Scale (~25 min)
- Exercise C: NAIRR Open Datasets (~25 min)
- Discussion + Wrap-Up (~10 min)

# About This Workshop Series

12 sessions • March–August 2026

# Workshop Series Overview

- **12 modular sessions** for researchers at U.S.-based institutions
- Three focus areas: **Cybersecurity**, **Edge AI**, **Autonomous Systems**
- Each session is standalone — attend any combination

**What you gain:**

- Hands-on experience with national-scale AI infrastructure
- Cross-institutional collaboration (Oakland University + WPI)
- Direct access to NAIRR compute for your ongoing research
- Mentorship and community connections

# Workshop Schedule

- **Mar 21** — Intro to NAIRR (today)
- **Apr 4** — Cybersecurity: Threats & Defenses
- **Apr 18** — Edge AI: Deployment & Optimization
- **May 2** — Autonomous Systems: Foundations
- **May 16** — Federated Learning for Privacy
- **May 30** — Adversarial ML & Robustness
- **Jun 13** — Reinforcement Learning in Autonomy
- **Jun 27** — AI for Network Security
- **Jul 11** — Embedded AI for IoT
- **Jul 25** — AI Safety & Alignment
- **Aug 8** — Multi-Agent Systems
- **Aug 22** — Capstone & Showcase

# Why NAIRR Matters for Your Research

The compute gap — and how to close it

# The Research Compute Bottleneck

- Training models, processing large datasets, and running experiments require compute that **most university labs cannot provide**
- Buying GPU clusters is cost-prohibitive; cloud credits expire; IT departments have constraints
- The result: researchers wait hours for local runs, compromise on dataset size, or abandon compute-intensive approaches entirely

**What takes an hour on your laptop might take 2 minutes on Jetstream2.**
**We will demonstrate this today.**

# NAIRR: What You Can Access Right Now, for Free

NAIRR is a shared national infrastructure connecting U.S. researchers to:

### Computation

- 9 HPC/cloud systems
- NVIDIA A100, H100, L40S GPUs
- Specialized accelerators (Cerebras, Groq, SambaNova)

### Data

- AI-ready datasets across scientific domains
- No download fees, no access barriers

### Models

- Pre-trained models (NASA, NVIDIA, domain-specific)
- 1,000+ models via Azure AI catalog
- 650+ NVIDIA models on HuggingFace

### Support

- This workshop series
- Monthly NAIRR office hours
- AI Unlocked training sessions

# Who Runs NAIRR?

- **Led by the National Science Foundation (NSF)**
- Partnership of **14 federal agencies**: DOE, NIH, DARPA, NIST, NASA, NOAA, USDA, and more
- **28 private-sector and nonprofit partners**: Microsoft, NVIDIA, Google, Amazon, OpenAI, HuggingFace, and more

**Timeline:**

- NAIRR Pilot launched January 2024
- Transitioning to the **NAIRR Operations Center**
- $35 million over 5 years

*NAIRR's four objectives: accelerate AI-powered discovery, expand the AI workforce, integrate public + private AI resources, and advance trustworthy AI.*

# NAIRR's Four Strategic Objectives

### 1. Accelerate Discovery

- Provide compute and data at the scale modern AI research demands
- Remove infrastructure barriers so researchers focus on the science

### 2. Expand the AI Workforce

- Train researchers, educators, and students across all institution types
- Ensure access is not limited to elite programs

### 3. Increase Capacity & Integration

- Connect public and private AI resources into one ecosystem
- Standardize access so switching between systems is straightforward

### 4. Advance Trustworthy AI

- Promote interpretability, safety, security, and fairness
- Support research into AI risks alongside AI capabilities

## NAIRR by the Numbers

**600+**

Research projects supported

**50**

U.S. states with active participants

**6,000+**

Researchers and students served

**9**

Major compute systems available

*This is not a theoretical program. Your peers across the country are actively using NAIRR for research and teaching.*

# What Researchers Are Doing with NAIRR

### Science & Engineering

- Climate modeling with physics-informed neural networks on Delta GPUs
- Drug discovery using protein structure prediction on Bridges-2
- Materials science simulations with high-entropy alloy datasets

### Education

- University courses using Jetstream2 classroom allocations for ML assignments
- Workshop series like this one, training researchers across institutions

### NLP & Computer Vision

- Fine-tuning large language models on domain-specific corpora
- Satellite image analysis using NVIDIA models from HuggingFace
- Training custom embedding models for scientific search

### Interdisciplinary

- Social science researchers analyzing large survey datasets
- Digital humanities projects processing text archives at scale

*These are not hypothetical — these projects are actively running on NAIRR systems right now.*

# Infrastructure Available to You

Compute, data, models, and more

# 9 Major Compute Systems

- **Jetstream2** (Indiana U) — Cloud VMs, GPUs, classrooms
- **Delta** (NCSA) — 800+ NVIDIA A100 GPUs
- **Bridges-2** (PSC) — GPU + CPU, large memory
- **Anvil** (Purdue) — 1,000+ CPU nodes
- **Frontera** (TACC) — #1 academic supercomputer
- **Lonestar6** (TACC) — A100 GPUs, general purpose
- **Neocortex** (PSC) — Cerebras wafer-scale engines
- **ANL AI Testbed** (Argonne) — Cerebras, Groq, SambaNova
- **ACES** (Texas A&M) — AMD MI250X GPUs

*Today's exercises use **Jetstream2**. The lecture covers what else is available for your larger-scale needs.*

# Jetstream2 — Cloud Computing for Research

On-demand virtual machines you control, with GPU options:

| Flavor | CPUs | RAM | GPU |
|---|---|---|---|
| m3.small | 4 | 15 GB | — |
| m3.medium | 8 | 30 GB | — |
| m3.large | 16 | 60 GB | — |
| m3.xl | 32 | 125 GB | — |
| g3.small | 8 | 30 GB | 1x NVIDIA A100/H100 |
| g3.large | 32 | 125 GB | 1x NVIDIA A100/H100 |

*Managed via the **Exosphere** portal. Instances take ~3.5 minutes to build. Access via Web Shell, Web Desktop, or SSH tunnel.*

# Why Jetstream2 Is the Entry Point

**For researchers:**

- Full root access — install anything you need
- Persistent instances that stay running between sessions
- GPU flavors (A100, H100) for deep learning
- SSH tunnel workflow — code on your machine, compute on NAIRR
- Attach storage volumes for large datasets

**For teaching:**

- Classroom allocations give all students identical environments
- No “it works on my machine” issues
- Web Desktop option for participants unfamiliar with CLI
- Scale to any class size without IT bottlenecks
- Students learn on real research infrastructure, not toys

*Other NAIRR systems (Delta, Frontera, etc.) require job schedulers and batch queues. Jetstream2 gives you an on-demand VM — the simplest on-ramp to national compute.*

# Free Models, Datasets, and Azure Credits

### NAIRR Free Resources

- nairrpilot.org/pilotresources
- Pre-trained models (some on HuggingFace, some require signup)
- Open datasets across scientific domains
- No allocation needed for many resources

### NVIDIA on HuggingFace

- 180+ datasets, 650+ models
- Permissive licenses

### Azure AI Model Catalog

- 1,000+ models (OpenAI, Meta, Mistral, etc.)
- $20M credit pool via CloudBank
- First-come, first-served

### Pre-installed Software

- PyTorch, TensorFlow, JAX, scikit-learn
- HuggingFace (Transformers, Datasets)
- NVIDIA CUDA/cuDNN/RAPIDS
- Conda/Miniforge, JupyterLab

# The HuggingFace Ecosystem — Your Research Toolbox

HuggingFace is tightly integrated with NAIRR. You will use it in all three exercises today.

### Datasets Hub

- 200,000+ datasets across every domain
- One line of code to load any dataset
- Text, vision, audio, tabular, scientific
- No registration required for most

### Model Hub

- Hundreds of thousands of pre-trained models
- NVIDIA models with permissive licenses
- NASA domain-specific models (we use one today)
- Fine-tune on your own data with `transformers` library

### Example — loading a dataset is this simple:

```
dataset = load_dataset("ag_news", split="train")  —  120K articles, ready to use
```

# Getting and Managing Allocations

What you can request — and how

# Allocation Types

| Type | Duration | Best For |
|---|---|---|
| **Start-Up** | 3 months | Exploring a new project, prototyping, benchmarking |
| **Research** | 12 months | Active, funded research with sustained compute needs |
| **Classroom / Education** | Semester-length | Teaching AI courses, student assignments, workshops |
| **Deep Partnerships** | Varies | Enhanced collaboration with resource providers |
| **Sandbox** | Short-term | Quick experimentation, demos |

**Portal:** submit-nairr.xras.org — Submit by the 15th, decision by end of the following month.

*You already have ACCESS accounts. If anyone needs help with their account, see the instructor after the session.*

# What Goes in an Allocation Request

A **Classroom / Education** request is a 3-page PDF (NSF-style formatting):

### Course Details

- Institution, course title, enrollment
- Timeline (which semester)
- How AI/compute integrates into the curriculum

### Resource Estimates

- GPU vs. CPU needs
- Storage requirements
- Software / frameworks
- Concurrent users expected

### Team Qualifications

- PI and co-PI experience
- Prior use of HPC or cloud compute
- Technical readiness

### Practical Tips

- Start-Up allocations require less justification — good for getting your feet wet
- Research allocations need a more detailed proposal but give 12 months of access
- Submit by the 15th → decision by end of next month

*Info: [nairrpilot.org/opportunities/education-call](nairrpilot.org/opportunities/education-call)*

# Today's Allocation

- We are using a **Classroom allocation** on Jetstream2
- Allocation: **NAIRR250048** (Oakland University / WPI)
- Provides: CPU and GPU instances, persistent storage

**What this means for today:**

- You will connect to a Jetstream2 instance and run exercises there
- Same notebooks run on your laptop *and* on Jetstream2
- Timing comparisons will show the compute advantage

*If you want this kind of access for your own research or courses, a Start-Up or Classroom allocation request is straightforward to submit.*

# Live Walkthrough

The NAIRR Portal and Jetstream2

# Portal Tour — Follow Along

Open these in your browser as we walk through them:

1. **nairrpilot.org** — Homepage, Getting Started pathways
2. **submit-nairr.xras.org/resources** — Resource catalog: browse compute, data, models
3. **nairrpilot.org/pilotresources** — Free models & datasets (note the NASA models, HuggingFace links)
4. **Exosphere Dashboard** — Our allocation, instance management

# Jetstream2 — How You Connect

| Method | What It Is | When to Use |
|---|---|---|
| **SSH Tunnel** | Work from your own machine; Jupyter runs on Jetstream2, appears in your local browser | Day-to-day research workflow (recommended) |
| **Web Desktop** | Full Linux desktop (XFCE) in your browser | When you need a GUI or self-contained environment |
| **Web Shell** | Terminal in your browser | Quick CLI tasks, package installation |
| **Console** | Serial console | Boot/network troubleshooting |

**Today's exercises use SSH tunnel.** Most researchers prefer this — you use your own browser, your own editor, your own shortcuts. The Web Desktop button is available on your instance page if you prefer a GUI.

# SSH Tunnel — The Day-to-Day Workflow

Three commands, and Jupyter running on Jetstream2 appears in your local browser:

| Step | What You Do | Where |
|---|---|---|
| **1** | `ssh -L 8888:localhost:8888 exouser@<ip>` | Your laptop terminal |
| **2** | `conda activate workshop && jupyter lab --no-browser --port=8888` | On the instance (via SSH) |
| **3** | Open `http://localhost:8888` | Your laptop browser |

### What this gives you:

- JupyterLab appears in *your* browser, but code runs on Jetstream2
- Use your own keyboard shortcuts, screen layout, and browser extensions
- Upload/download files through JupyterLab's file browser
- Close your laptop → reconnect later — the instance keeps running

# What We'll Do in the Exercises

| Exercise | What You'll Build | Key Metric |
|---|---|---|
| **A: Models + HuggingFace** | Semantic search engine using a NASA sentence transformer | Encoding time: 20 → 5,000 passages |
| **B: Bulk Data Processing** | Full ML pipeline on 500K rows — features, PCA, grid search | Grid search time across all CPU cores |
| **C: Open Datasets** | Text classifier on 120K news articles from HuggingFace | TF-IDF + training time at scale |

**Every exercise includes a timing comparison table.** Run each notebook locally, then run the same notebook on Jetstream2. The difference is the research productivity gain NAIRR provides.

*All three notebooks are standalone — if time is short, do any subset. They work on your laptop, on Jetstream2, on Colab, or on any NAIRR system.*

# Key Takeaways Before We Begin

- **NAIRR gives you free compute, data, and models** for research — no hardware purchases, no expiring cloud credits
- **Jetstream2 is the entry point** — cloud VMs with GPU options, accessible via SSH from your own machine
- **Timing matters:** the exercises will show concrete speedups between your laptop and Jetstream2
- **Allocations are straightforward** — a Start-Up request gets you access in weeks
- **Everything is portable:** the same `.ipynb` notebooks run locally, on Jetstream2, on Colab, or on any NAIRR system

# Let’s Get Started

First: connect to Jetstream2, then Jupyter crashcourse + exercises

# Getting Set Up

Local tools, Jetstream2 connection, Jupyter crashcourse

# Quick Check — What Do You Have?

Raise your hand (or type in chat):

- **Do you have VS Code installed?**
- **Do you have Python installed?** (any version)
- **Can you open a terminal?** (PowerShell, Terminal, Git Bash, CMD)

If you answered **no** to any of these — no worries. The next two slides walk through getting everything installed. We'll have a **breakout room with helpers** to get everyone set up.

*If you already have all three, you can skip ahead or help a neighbor.*

# Install VS Code + Extensions

## 1. Download VS Code

code.visualstudio.com

- Windows, macOS, or Linux
- Free, lightweight, widely used in research
- Run the installer, accept defaults

## 2. Install Extensions

Open VS Code → click the **Extensions** icon (left sidebar) → search and install:

1. **Python** (by Microsoft)
2. **Jupyter** (by Microsoft)

That's it — VS Code can now open `.ipynb` notebooks and run Python code.

*Already have VS Code? Just make sure the Python and Jupyter extensions are installed.*

# Install Python

## Option A: python.org

python.org/downloads

1. Download Python 3.12 (or latest 3.x)
2. **Windows:** check **"Add python.exe to PATH"** during install
3. macOS / Linux: usually pre-installed; installer if not

## Option B: Anaconda / Miniconda

docs.anaconda.com/miniconda

- Comes with `conda` for environment management
- Good if you plan to use multiple Python versions
- Miniconda is the lightweight version

**Verify it works** — open a terminal and type:

```
python --version       # should print Python 3.x.x
pip --version          # should print pip 2x.x
```

*If `python` doesn't work, try `python3`. On Windows, you may need to restart your terminal after installing.*

# Open a Terminal

You'll use a terminal to SSH into Jetstream2. Here's how to open one:

| OS | Terminal App | How to Open |
|---|---|---|
| **Windows** | PowerShell or Windows Terminal | Start menu → type `powershell` or `terminal` |
| **macOS** | Terminal | Spotlight (Cmd+Space) → type `terminal` |
| **Linux** | Terminal / Konsole / GNOME Terminal | Ctrl+Alt+T (most distros) |
| **VS Code** | Integrated terminal | Ctrl+` (backtick) — works on all platforms |

**Test it:** Type `ssh` and press Enter. You should see usage info, not "command not found."

***Windows note:*** *If* `ssh` *is not found, install Git for Windows — it includes Git Bash with SSH built in.*

# Setup Breakout Room

If you need help installing VS Code, Python, or getting your terminal working — **join the Setup Breakout Room now.**

Helpers are standing by to walk you through it.

**Once you have:**

- VS Code with Python + Jupyter extensions
- Python 3.x working in your terminal ( `python --version` )
- SSH available ( `ssh` shows usage info)

… you're ready. Come back to the main room and we'll connect to Jetstream2.

*If everything is already working, sit tight — we'll continue in a few minutes.*

# Scenario A: Your Own Instance

Create and configure a Jetstream2 VM from scratch

# A1: Create a Jetstream2 Instance

1. Go to jetstream2.exosphere.app — log in with your ACCESS credentials
2. Select allocation **NAIRR250048**
3. **Create > Instance > Ubuntu 24.04**
4. Pick your flavor:

| Flavor | CPUs | RAM | Good For |
|---|---|---|---|
| m3.small | 4 | 15 GB | Light work, testing |
| **m3.medium** | 8 | 30 GB | Today's exercises, most research |
| m3.large | 16 | 60 GB | Larger datasets, more parallelism |
| g3.small | 8 + GPU | 30 GB | Deep learning (A100/H100) |

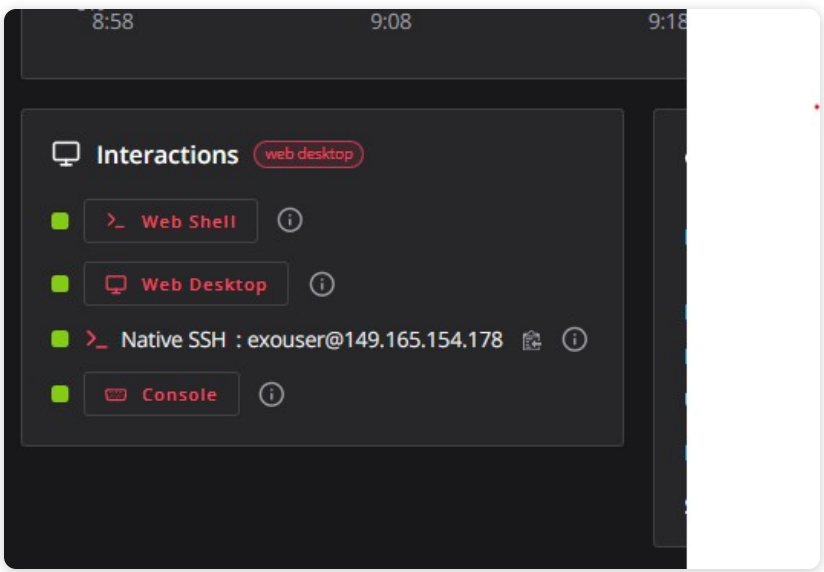
5. Enable **Web Desktop** > click **Create**
6. Wait ~3.5 minutes for "Ready"

# A2: Find Your Connection Details

Click on your instance name in Exosphere. You need two things:

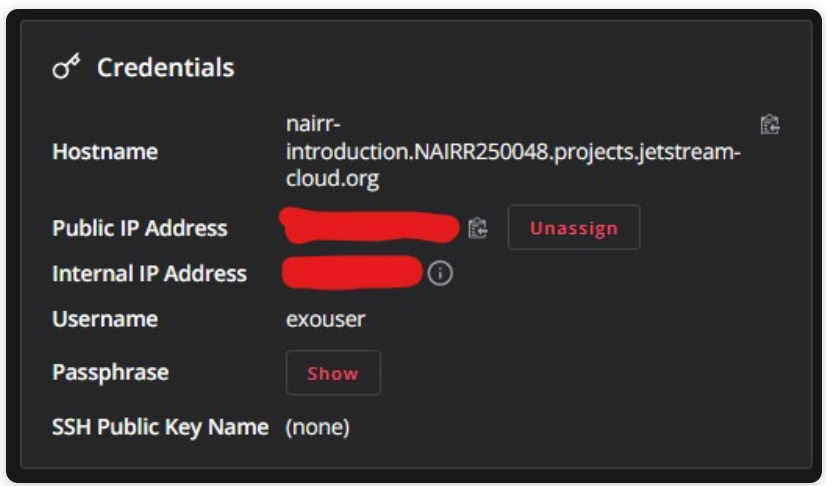
## 1. SSH Address

Under **Interactions**, find **Native SSH** — it shows `exouser@<ip>`. Click the copy icon.




## 2. Passphrase

Under **Credentials**, click **Show** next to Passphrase. This is your SSH password — keep it private.




*Exosphere auto-generates a passphrase for each instance. You'll type it when SSH prompts for a password.*

# A3: Connect via SSH Tunnel

In your terminal:

```
ssh -L 8888:localhost:8888 exouser@<your-instance-ip>
```

Enter the passphrase when prompted. You're now on the Jetstream2 instance.

### What this does:

- Connects you to the remote machine via SSH
- The `-L 8888:localhost:8888` part creates a **tunnel** — port 8888 on your laptop forwards to port 8888 on the instance
- This is how JupyterLab (running remotely) will appear in your local browser

# A4: Set Up the Python Environment

The instance has system Python but no conda or JupyterLab. Jetstream2 provides conda via **modules**:

```
module load miniforge                        # makes conda available
conda create -n workshop python=3.12 -y      # create an isolated environment
conda init && source ~/.bashrc               # one-time shell setup
conda activate workshop                      # switch into the environment
pip install jupyterlab ipykernel             # install Jupyter + kernel support
pip install -r requirements.txt              # install exercise dependencies
```

**Then register the kernel** so JupyterLab can find it:

```
python -m ipykernel install --user --name workshop \
    --display-name "Workshop (Python 3.12)"
```

# A5: Launch JupyterLab

**On the instance** (inside your SSH session):

```
conda activate workshop
jupyter lab --no-browser --port=8888
```

You'll see output like:

```
http://localhost:8888/lab?token=abc123...
```

**On your local machine**, open that full URL (with token) in your browser.

Because of the SSH tunnel, `localhost:8888` on your machine connects to port 8888 on the Jetstream2 instance. JupyterLab is running *there*, but you see it *here*.

*Copy the full URL including the token — JupyterLab needs it the first time you connect.*

# Scenario B: Shared Instance

The instructor set up an account for you — just connect

# B1: Your Credentials

You have your own **Linux account** on a shared Jetstream2 instance. The instructor will share your credentials:

| Field | Example |
|---|---|
| **Username** | `user07` |
| **Password** | `nairr2026` |
| **Port** | `8894` |
| **Instance IP** | `149.165.152.105` |

You have your own home directory, your own JupyterLab process, and all 4 notebooks pre-loaded. Other participants cannot see your files.

*Check the chat or spreadsheet the instructor shared for your specific username and port number.*

# B2: Connect & Open JupyterLab

**Step 1:** Open your terminal and SSH in:

```
ssh -L 8894:localhost:8894 user07@149.165.152.105
```

Replace `user07`, `8894`, and the IP with your values. Enter the password when prompted.

**Step 2:** Open your browser to:

```
http://localhost:8894/lab
```

No token needed — JupyterLab opens directly to your home directory with all notebooks ready.

**If JupyterLab doesn't load:** In your SSH session, run:

```
bash ~/start_jupyter.sh
```

*Then refresh the browser. This restarts your JupyterLab process on your assigned port.*

# It’s Just a Machine

Your Jetstream2 instance is a **full Ubuntu 24.04 Linux machine**. You have root access (Scenario A) or your own user account (Scenario B). You can:

- `ssh` in and use the terminal directly
- Install any software: `apt install`, `pip install`, `conda install`
- Run Python scripts, shell scripts, Docker containers — anything
- Use the **Web Desktop** button in Exosphere for a full graphical Linux desktop

**Today we use JupyterLab** because it’s ideal for interactive, exploratory work. But this machine is yours to use however fits your research.

***For research labs:*** *a single instance can serve your entire group — each researcher gets their own Linux account, they share data and the same environment. That’s exactly how the shared instance works today.*

# Python Environments — Why They Matter

A **Python environment** is an isolated set of packages and a Python version. Think of it as a self-contained workspace for a project.

## Why use them?

- **No conflicts** — Project A needs PyTorch 2.1, Project B needs 2.4? No problem.
- **Reproducible** — share `requirements.txt` and anyone can recreate your setup
- **Clean** — delete the env when you're done, system Python is untouched

## Creating one

```
conda create -n myproject python=3.12
conda activate myproject
pip install numpy pandas torch
```

Today we use `conda activate workshop` — a pre-built environment with all exercise dependencies.

*If you're used to R or MATLAB, this is new. In Python it's standard practice — every project gets its own environment.*

# Kernels — Where Your Code Runs

A **kernel** is the Python process that executes your notebook code. When you pick a kernel, you're choosing *which Python environment* runs the cells.

## In JupyterLab

- Kernel name appears in the **top-right** of the notebook
- Click it to switch to a different environment
- If it says `workshop`, your code runs in the `workshop` conda env

## In VS Code

- **"Select Kernel"** button in the top-right
- Lists all detected Python environments (conda, venv, system)
- Pick `workshop` for Jetstream2, or your local env for laptop runs

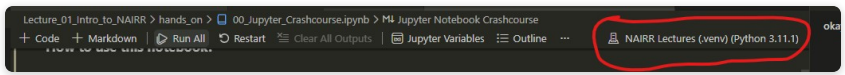
**Key idea:** Same notebook, different kernel = code runs in a different place with different packages. This is how you switch between local and Jetstream2.

*The crashcourse notebook includes a cell that shows which kernel/environment you're using.*

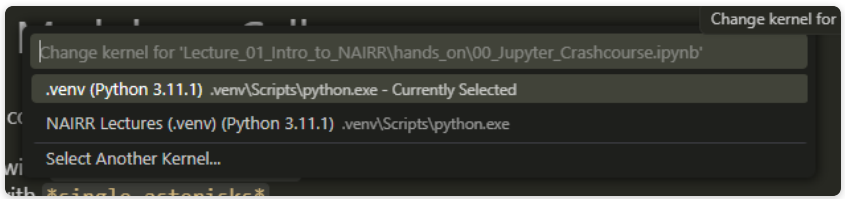
# VS Code → Jetstream2 (Remote Kernel)

Use **VS Code on your laptop** to edit notebooks while **code runs on Jetstream2**. Same SSH tunnel, familiar editor.

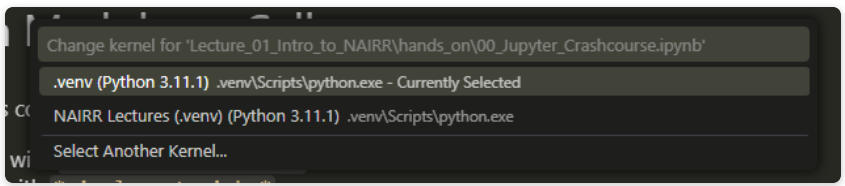
**1.** See the active kernel (top-right) — click it



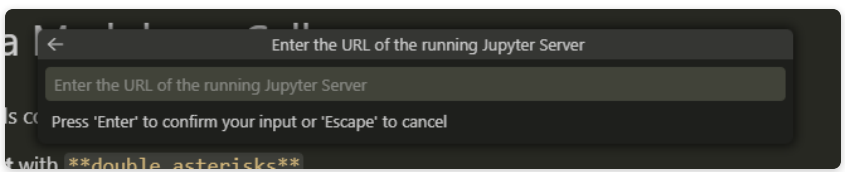

**2.** Click **"Select Another Kernel..."**




**3.** Choose **"Existing Jupyter Server..."**




**4.** Paste the `localhost:8888` URL with token




**Then pick Workshop (Python 3.12)** — now every cell runs on Jetstream2. Swap back to a local kernel anytime.

# Running Notebooks Locally

For the timing comparisons later, you'll also run notebooks on your own machine:

## VS Code (recommended)

1. Same Jupyter extension you installed earlier
2. **Select Kernel** → pick your **local** Python environment
3. Run the same notebook — now it runs on your laptop

*If you don't have the exercise dependencies locally, run `pip install -r requirements.txt` first.*

## Other options

- **JupyterLab in browser:** `pip install jupyterlab`, then `jupyter lab`
- **Google Colab:** upload the `.ipynb` — zero setup

*You don't need to set this up right now. We'll do the exercises on Jetstream2 first, then you can try locally after the session.*

# Jupyter Notebook Crashcourse

The tool you'll use for every exercise

# What Is a Jupyter Notebook?

- A **browser-based interactive environment** for writing and running code
- Mix **code, formatted text, and output** (including plots) in one document
- Python-based, but also supports R, Julia, and other languages
- File format: `.ipynb` — a JSON file that captures code, output, and narrative together

**The name: Ju**lia + **Py**thon + **R** — but today we use Python exclusively.

*Jupyter is the standard tool for reproducible computational research. If you've used MATLAB's Live Editor or Mathematica notebooks, the concept is similar.*

# Why Researchers Use Jupyter

**For your own work:**

- Inline visualizations — plots appear right below the code
- Iterative exploration — run one cell at a time, inspect results, adjust
- Mix narrative and code — document your reasoning alongside the analysis
- Reproducible — anyone can re-run your notebook and get the same results

**For collaboration:**

- Share `.ipynb` files with colleagues — they see code + results
- GitHub renders notebooks in the browser
- Export to PDF or HTML for reports
- Common in publications as supplementary material

*This is not just a teaching tool. Researchers at CERN, NASA, and major tech companies use Jupyter notebooks in production research workflows.*

# Where You Can Run Jupyter

| Option | How | When to Use |
|---|---|---|
| **VS Code** | Install the Jupyter extension, open `.ipynb` files | Day-to-day local work, integrated with your editor |
| **JupyterLab (browser)** | `pip install jupyterlab`, then `jupyter lab` | Standalone environment, full-featured interface |
| **Google Colab** | Open colab.research.google.com | Quick experiments, free GPU, no setup |
| **Jetstream2 / NAIRR** | SSH tunnel + `jupyter lab` on the instance | Large-scale compute, persistent environment |

**Right now:** you're using JupyterLab on Jetstream2. All four options run the same `.ipynb` files — your notebook is portable.

# The JupyterLab Interface

**Layout:**

- **File browser** on the left — navigate, upload, create files
- **Notebook area** in the center — your cells live here
- **Toolbar** at the top — run, stop, restart, cell type dropdown
- **Kernel indicator** top-right — circle is filled when busy

**Two cell types:**

- **Code cells** — write Python, press `Shift+Enter` to run, output appears below
- **Markdown cells** — formatted text (headers, bold, links, lists). Double-click to edit, `Shift+Enter` to render

**Key shortcuts:**

- `Shift+Enter` — run cell, move to next
- `Ctrl+Enter` — run cell, stay in place

# Open the Crashcourse Notebook

## Notebook: `00_Jupyter_Crashcourse.ipynb`

Open it in JupyterLab and follow along. We'll work through it together, flipping back to these slides as needed.

### What you'll practice:

- Running code cells and editing markdown cells
- Python basics: variables, lists, functions, imports
- Installing packages with `!pip install`
- Working with DataFrames and inline plots
- Keyboard shortcuts that speed up your workflow
- Restart & Run All for reproducibility

*~15–20 minutes. The last cell verifies your Jetstream2 connection.*

# Verify Your Connection

The last cell of the crashcourse notebook runs a quick check:

```
import platform, os
print(f"Hostname:  {platform.node()}")
print(f"CPU cores: {os.cpu_count()}")
print(f"OS:        {platform.platform()}")
!nvidia-smi 2>/dev/null || echo "CPU instance (no GPU)"
```

### What to look for:

- **Hostname** — should be your Jetstream2 instance name, not your laptop
- **CPU cores** — 8 for m3.medium (vs. likely 4–8 on your laptop)
- **GPU** — if on a g3 instance, `nvidia-smi` will show the A100/H100

### Everyone seeing their instance? Great — let's move to the exercises.

# Hands-On Exercises

Local vs. Jetstream2 — see the difference

# The Timing Framework

Every exercise uses the same pattern to measure performance:

```
import time

start = time.perf_counter()
# ... your operation ...
elapsed = time.perf_counter() - start
print(f"Completed in {elapsed:.2f} seconds")
```

### Your workflow:

1. Run each notebook on **Jetstream2** today — record the timing numbers
2. Later, run the same notebook on **your laptop** — record the timing numbers
3. Compare — the difference is the **research productivity gain** NAIRR provides

*Each notebook includes a timing comparison table for you to fill in.*

# Exercise A: NAIRR Models + HuggingFace

**Notebook:** `01_NAIRR_Models_and_HuggingFace.ipynb`

- Browse NAIRR's free models at [nairrpilot.org/pilotresources](nairrpilot.org/pilotresources)
- Load a **NASA sentence transformer** ( `nasa-impact/nasa-smd-ibm-st-v2` ) — 125M parameters, fine-tuned on NASA Science Mission Directorate data
- Encode scientific text, compute semantic similarity, find related passages
- Scale from 20 passages to 1,000 — **measure timing at each step**

**Key question:** How does encoding speed scale with corpus size, and how does your laptop compare to Jetstream2?

# Exercise B: Bulk Data Processing at Scale

**Notebook:** `02_Bulk_Data_Processing.ipynb`

- Generate a **500,000-row synthetic dataset** (~200 MB) with 50 features
- Run a full data pipeline: feature engineering, PCA, cross-validated model training
- Use `n_jobs=-1` to leverage **all available CPU cores**
- Run hyperparameter grid search — the kind of operation that **bottlenecks local machines**

**Key question:** Your laptop shows N CPU cores. Jetstream2 m3.medium has 8. How does that translate to wall-clock time on parallelizable workloads?

# Exercise C: NAIRR Open Datasets

**Notebook:** `03_NAIRR_Open_Datasets.ipynb`

- Browse NAIRR's free datasets and the HuggingFace dataset hub
- Pull the **AG News dataset** (120,000 news articles, 4 categories) directly into the notebook
- Vectorize with TF-IDF, train a text classifier, evaluate performance
- **Measure timing** for download, vectorization, and training

**Key question:** With one line of code, you can load any of 200,000+ datasets from HuggingFace. How does access to large, curated data change what is feasible in your research?

# Resources & Links

### NAIRR

- nairrpilot.org
- Resource Catalog
- Free Models & Datasets
- Classroom Allocations

### Jetstream2

- Exosphere Portal
- Documentation

### Data & Models

- HuggingFace Datasets
- NVIDIA on HuggingFace
- NASA Sentence Transformer

### Accounts

- ACCESS (access-ci.org)
- CloudBank

# What's Next

- **Session 2:** Cybersecurity in AI — Threats and Defenses (April 4, 2026)

**Before next time:**

- Explore the resource catalog for resources relevant to your research
- Consider submitting a Start-Up allocation request for your own project
- Re-run today's notebooks locally and compare your timing numbers
- Join the NAIRR community — attend monthly office hours

**Thank you!**

Oakland University & Worcester Polytechnic Institute

NAIRR Workshop Series 2026